



University of Piraeus

School of Information and Telecommunications Technologies

Department of Digital Systems

Level: Postgraduate Program of Study

Network Security

Case 6

Supervisor: Prof. Christos Xenakis

Kalaitzidis Ioannis

ioannis\_kalaitzidis@ssl-unipi.gr

Piraeus

26/11/2025



## **Abstract**

Carrying out a DNS Spoofing attack targeting the DNS server infrastructure implemented in the previous exercise. Using the Ettercap tool on Kali Linux, MITM position was achieved via ARP Poisoning. Then, through the dns\_spoof add-on, the victim's traffic was successfully redirected to a malicious webpage hosted by the Apache server, rather than the legitimate destination.

## Contents

|                              |    |
|------------------------------|----|
| Introduction.....            | 1  |
| Practical piece of work..... | 1  |
| Bibliography .....           | 10 |

# **Introduction**

DNS (Domain Name System) is one of the cornerstones of the operation of the internet, undertaking the assignment of website names (domain names) to IP addresses. Due to its criticality, it is a frequent target for cyberattacks. One of the most prevalent attack techniques is DNS Spoofing.

In this paper we will implement such an attack and use as a basis the infrastructure developed in the previous exercise (Case 5), where a legitimate DNS server (Ubuntu/BIND9) was installed and configured.

The purpose of the exercise is to understand the mechanisms of MITM attack. We will use the Kali Linux operating system and the Ettercap tool to perform ARP Poisoning, intervening between the victim (Windows 10) and the DNS server. The goal is to lead the victim to a controlled page (kalaitzon.gr) hosted on the attacker's Apache Web Server.

## **Practical piece of work**

First we start by finding the IP addresses with the ip a command in kali linux and ubuntu and ipconfig for windows 10. They are the same IPs from the previous exercise where we created DNS Server.

```

kali@kali:~$ sudo systemctl status bind9
[sudo] password for kali@kali:
● named.service - BIND Domain Name Server
   Loaded: loaded (/usr/lib/systemd/system/named.service; enabled; preset: enabled)
   Active: active (running) since Sat 2025-11-22 02:41:16 EET; 3 days ago
     Docs: man:named(8)
    Main PID: 1189 (named)
      Status: "running"
        Tasks: 14 (limit: 9374)
      Memory: 31.8M (peak: 31.8M)
         CPU: 11.296s
    CGroup: /system.slice/named.service
            └─1189 /usr/sbin/named -f -u bind

Nov 23 00:59:06 ubuntudesktop1.kalaitzon named[1189]: validating CK8P03HC874JREF7EFH8430QVIT88SM.com/NSEC3: no valid signature found
Nov 23 00:59:06 ubuntudesktop1.kalaitzon named[1189]: validating com/SDA: verify failed due to bad signature (keyid=46539): RRSIG validity period has not begun
Nov 23 00:59:06 ubuntudesktop1.kalaitzon named[1189]: validating com/SDA: no valid signature found
Nov 23 00:59:06 ubuntudesktop1.kalaitzon named[1189]: validating TCQ78V56HPBSM9C06KGF1900GR176Q0.com/NSEC3: verify failed due to bad signature (keyid=46539): RRSIG validity period has not begun
Nov 23 00:59:06 ubuntudesktop1.kalaitzon named[1189]: validating TCQ78V56HPBSM9C06KGF1900GR176Q0.com/NSEC3: no valid signature found
Nov 23 00:59:06 ubuntudesktop1.kalaitzon named[1189]: no valid RRSIG resolving 'microsoft.com/OS/IN': 192.55.83.30#53
Nov 23 00:59:06 ubuntudesktop1.kalaitzon named[1189]: broken trust chain resolving 'oneocsp.microsoft.com/A/IN': 8.8.8.8#53
Nov 25 18:20:15 ubuntudesktop1.kalaitzon named[1189]: _default: sending trust-anchor telemetry query '_ta-4f66-9728/NULL'
Nov 25 18:20:15 ubuntudesktop1.kalaitzon named[1189]: managed-keys-zone: Key 26326 for zone . is now trusted (acceptance timer complete)
Nov 25 18:20:15 ubuntudesktop1.kalaitzon named[1189]: managed-keys-zone: Key 38696 for zone . is now trusted (acceptance timer complete)

kali@kali:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens32: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:0c:29:f5:51:1f brd ff:ff:ff:ff:ff:ff
    inet 192.168.137.129/24 brd 192.168.137.255 scope global dynamic noprefixroute ens32
        valid_lft 1276sec preferred_lft 1276sec
    inet6 fe80::36ed:f8e6:63b6:348c/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
kali@kali:~$

```

```

kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:0c:29:f5:51:1f brd ff:ff:ff:ff:ff:ff
    inet 192.168.137.129/24 brd 192.168.137.255 scope global dynamic noprefixroute eth0
        valid_lft 1276sec preferred_lft 1276sec
    inet6 fe80::36ed:f8e6:63b6:348c/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: br-480f97c7b15a: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:2d:f0:d0:c7 brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.1/16 brd 172.18.255.255 scope global br-480f97c7b15a
        valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:2d:f0:d0:c7 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
(kali@kali)-[~]
$

```

```

C:\ Command Prompt
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Client_Win10>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet0:

    Connection-specific DNS Suffix  . : localdomain
    Link-local IPv6 Address . . . . . : fe80::143f:92cd:f6cf:6529%5
    IPv4 Address. . . . . : 192.168.137.130
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.137.2

C:\Users\Client_Win10>
```

**Attacker (Kali):** 192.168.137.128

**DNS Server (Ubuntu):** 192.168.137.129

**Victim (Windows 10):** 192.168.137.130

```

(kali@kali)-[~]
$ sudo systemctl start apache2
[sudo] password for kali:

(kali@kali)-[~]
$ sudo nano /var/www/html/index.html

(kali@kali)-[~]
$
```

With the mandate **sudo systemctl start apache2** We convert the user (Kali Linux-attacker) into a server to be able to host websites. We do this to "serve" the edited page by us to the victim. The page will only bear the title "YOU HAVE BEEN HACKED". That's why we use apache2. With the next command (**sudo nano /var/www/html/index.html**) we open the default root directory where Apache searches to find the web page files. We notice the following image when we open the file:

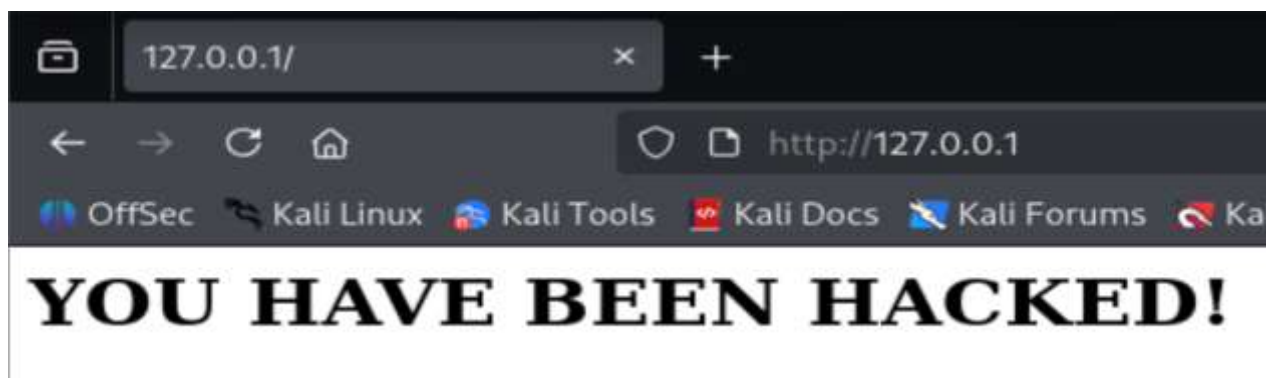
```
kali@kali: ~  
Session Actions Edit View Help  
GNU nano 8.6 /var/www/html/index.html  
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">  
<html xmlns="http://www.w3.org/1999/xhtml">  
  <head>  
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />  
    <title>Apache2 Debian Default Page: It works</title>  
    <style type="text/css" media="screen">  
      * {  
        margin: 0px 0px 0px 0px;  
        padding: 0px 0px 0px 0px;  
      }  
  
      body, html {  
        padding: 3px 3px 3px 3px;  
  
        background-color: #D0D0E2;  
  
        font-family: Verdana, sans-serif;  
        font-size: 11pt;  
        text-align: center;  
      }  
  
      div.main_page {  
        position: relative;  
        display: table;  
  
        width: 800px;  
  
        margin-bottom: 3px;  
        margin-left: auto;  
        margin-right: auto;  
        padding: 0px 0px 0px 0px;  
  
        border-width: 2px;  
        border-color: #212738;  
        border-style: solid;  
  
        background-color: #FFFFFF;  
      }  
    </style>  
  </head>  
</html>
```

We'll delete all content and leave only the `<h1> YOU HAVE BEEN HACKED! </h1>`. It is not a necessary process, but through it we show a tangible result of this attack.

```
kali@kali: ~  
Session Actions Edit View Help  
GNU nano 8.6 /var/www/html/index.html *  
<h1> YOU HAVE BEEN HACKED! </h1>
```

We save it by pressing Ctrl+O and Enter and leave the document by pressing Ctrl+X. To check that the page is working we will type in our browser the IP 127.0.0.1. This address is the most famous IP and is essentially localhost. To check if the apache we just set up on the attacker is working properly, we ran the website on the same host, without going to the network. We notice that the website is functional and it remains to move on to the next part: Ettercap





With the mandate **sudo nano /etc/ettercap/etter.dns** open the configuration file of the dns\_spoof, the Ettercap tool. In other words, etter.dns includes the domains that we want to "break" and send to our victim.

```
(kali@kali)-[~]  
$ sudo nano /etc/ettercap/etter.dns
```

After opening the file, we will write the following, mandatory but also optional:

- **kalaitzon.gr**: This line catches the case when someone writes the name plain, with nothing in front of it
- **\*.kalaitzon.gr**: Means "anything" before kalaitzon.gr. E.g. if the user writes mail.kalaitzon.gr, admin.kalaitzon.gr or anything else
- **www.kalaitzon.gr**: It is the classic case where we write www. Normally the previous case covers it but to be sure we state it separately

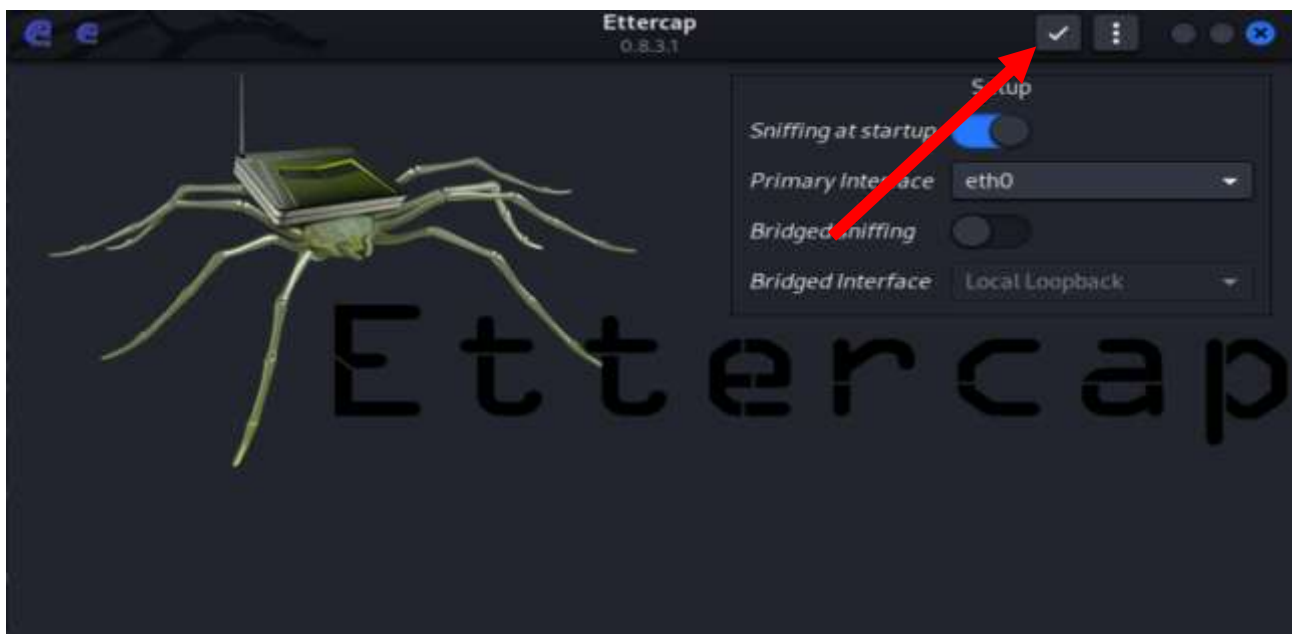
```
# or for TXT query (value must be wrapped in double quotes):  
# google.com TXT "v=spf1 ip4:192.168.0.3/32 ~all" [TTL]  
  
kalaitzon.gr      A      192.168.137.128  
*.kalaitzon.gr   A      192.168.137.128  
www.kalaitzon.gr A      192.168.137.128  
  
NOTE: the wilddcarded hosts can't be used to poison the PTR requests  
so if you want to reverse poison you have to specify a plain  
host. (look at the www.microsoft.com example)
```

In this way we say that if another user asks the network about the website kalaitzon.gr for example, then instead of feeding it back to the Ubuntu IP as a DNS

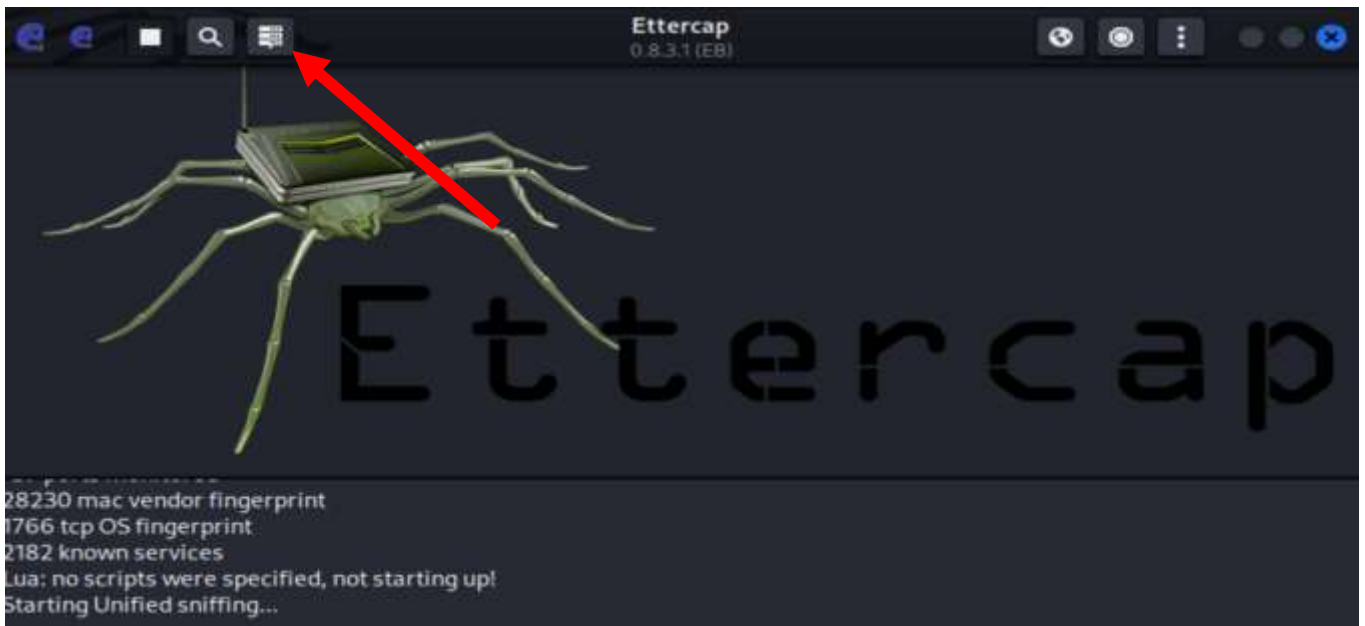
server, we feed it back to the attacker's IP (kali linux) without the victim knowing until he physically sees the message on his screen («YOU HAVE BEEN HACKED!»).

```
(kali@kali)-[~]  
$ sudo ettercap -G  
ettercap 0.8.3.1 copyright 2001-2020 Ettercap Development Team
```

Finally, we give administrator (root) permissions to monitor the network as MITM with the Ettercap tool. In addition, with -G we open a graphical interface to run the necessary actions in this way instead of writing them. So in a separate window we notice that the network interface is properly configured on the kali linux network card and for this we proceed normally to the next step.

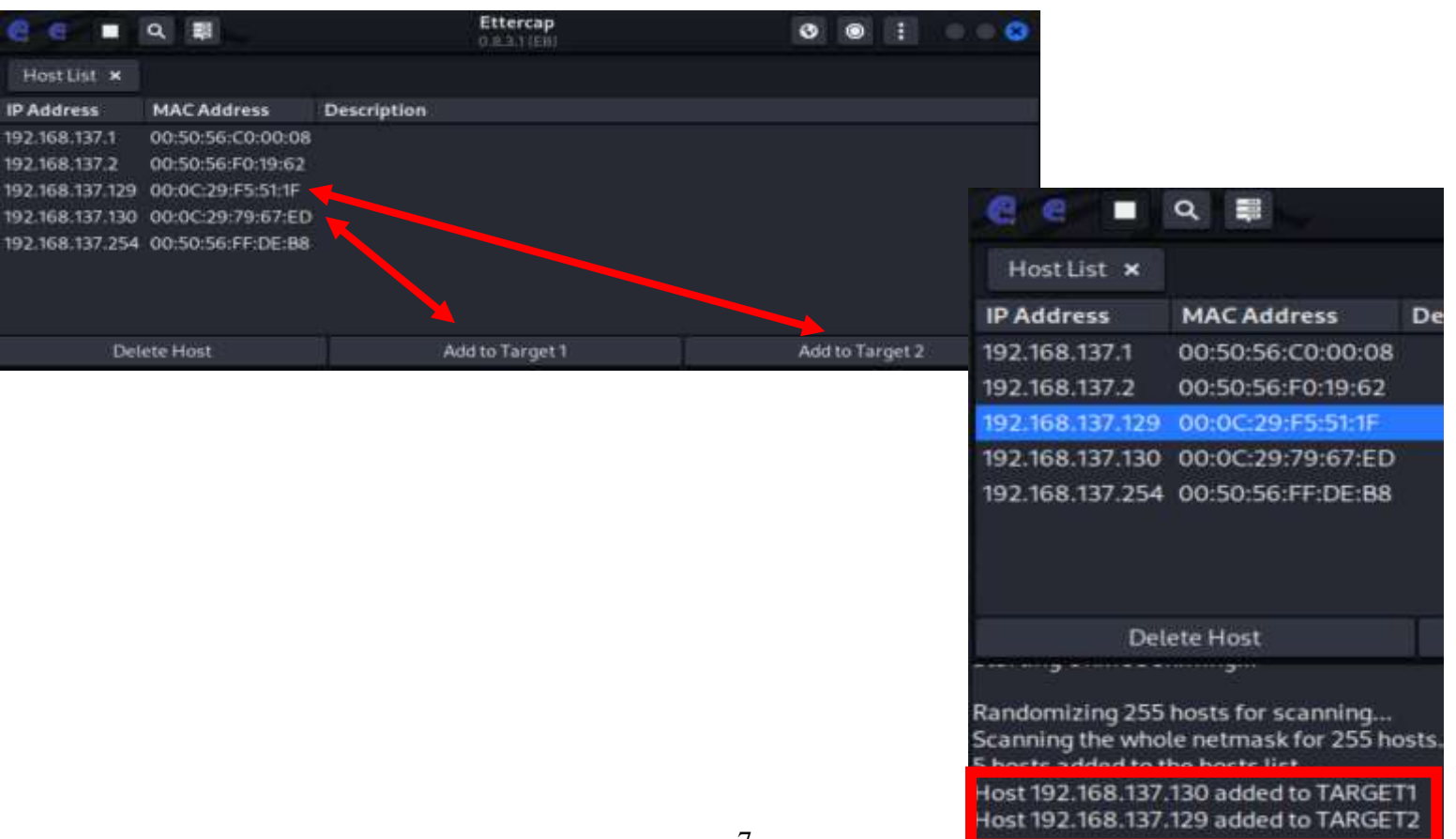


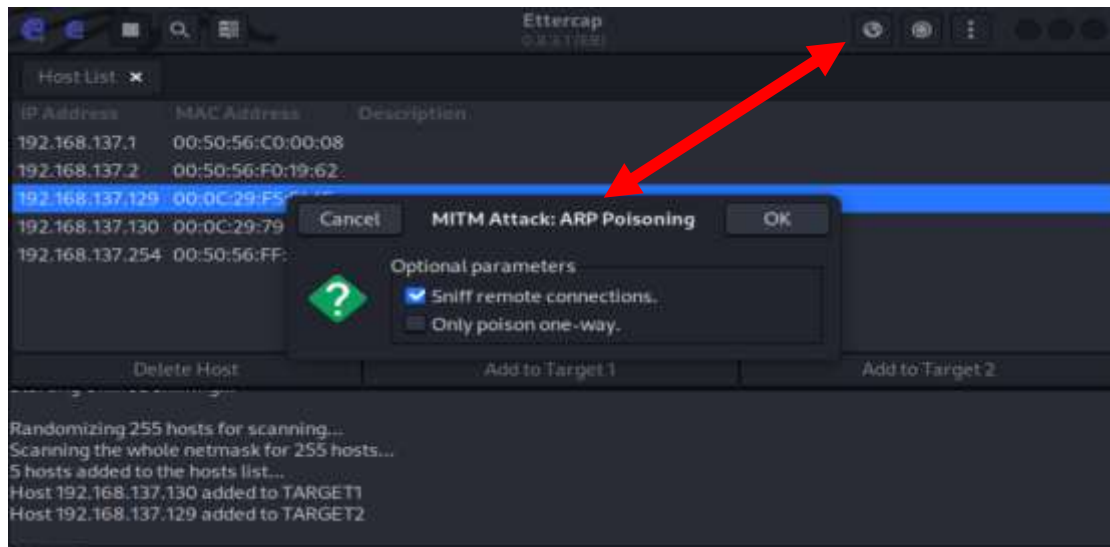
With the option indicated by the arrow, Ettercap instructed the network card (eth0) to enter Promiscuous Mode. That is, from where the network card only read the packets intended for the specific host (kali linux), it now reads everything that moves. In other words, it monitors network traffic.



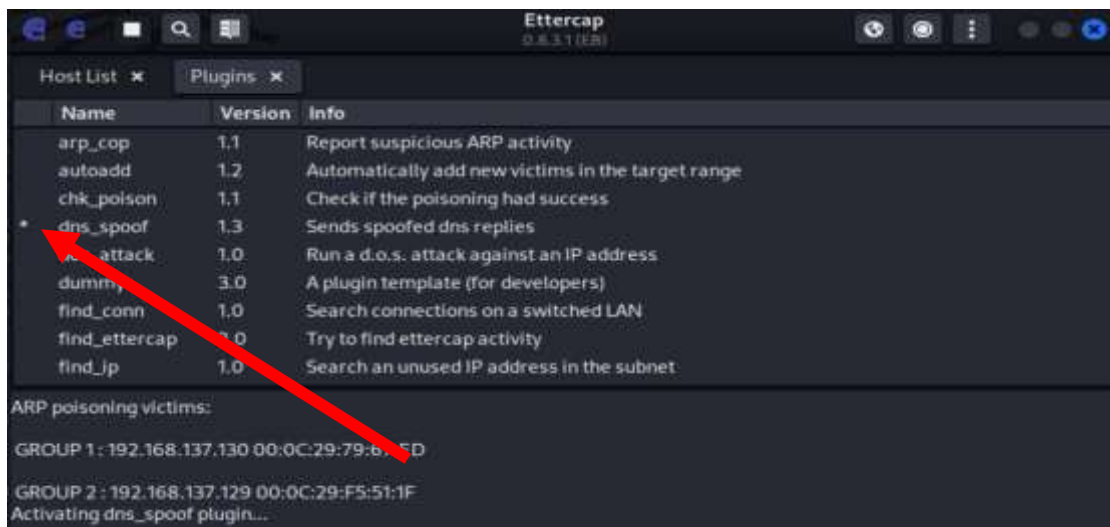
By selecting this icon we see that the scanning process is complete and shows all hosts detected by Ettercap on the network. We see the IP of both the victim and the DNS server.

We will choose our targets, specifically Windows 10 as target1 and Ubuntu DNS server as target2, as shown in the image below.



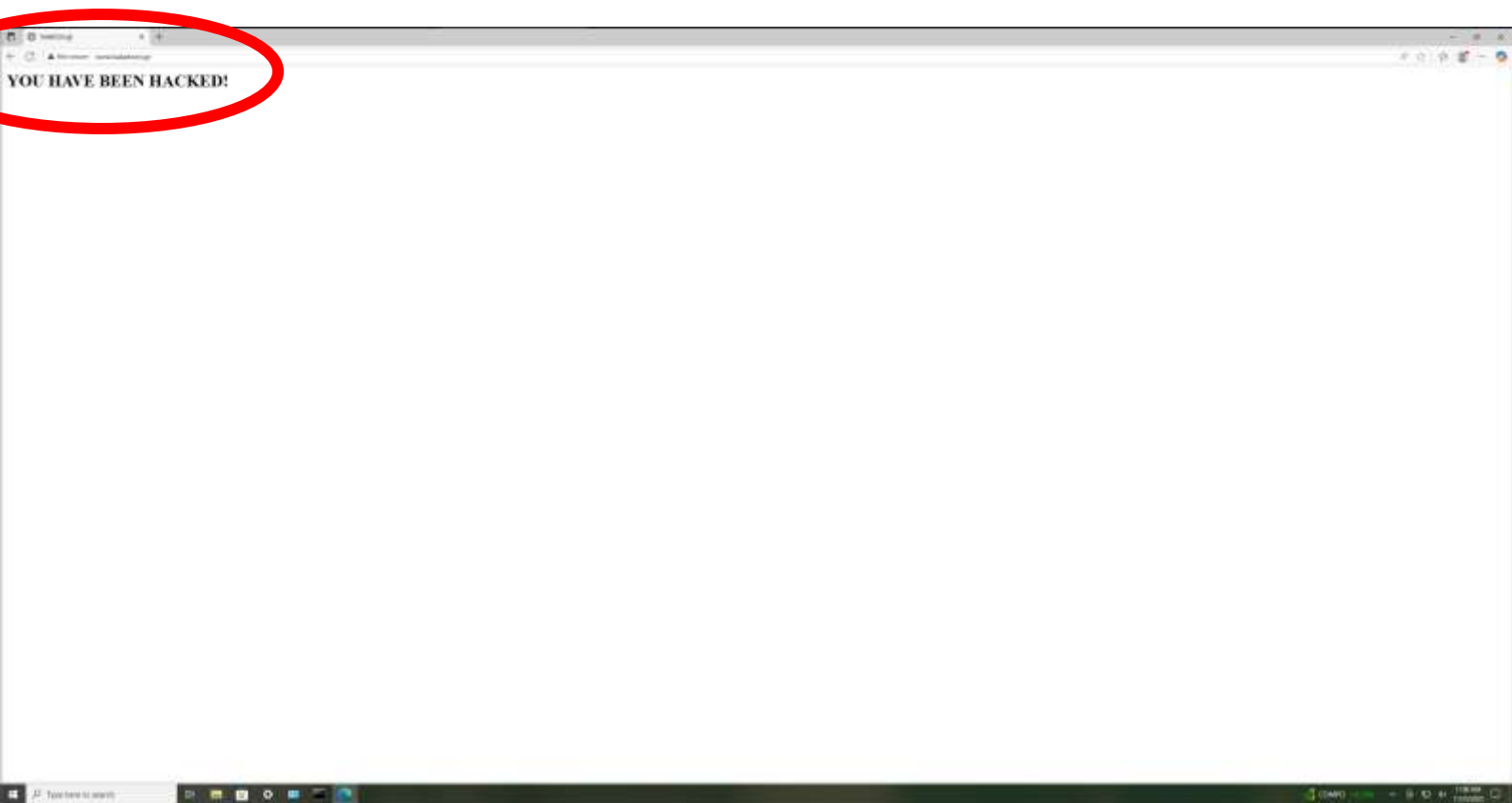


By selecting the globe icon and activating ARP Poisoning, we change the flow of information on the network. Specifically, before the attack, communication was immediate. That is, the victim (Windows 10) sent the DNS requests directly to the DNS server (Ubuntu). Now, with ARP Poisoning, we interfere as MITM. The tool convinces the victim that we are the server.



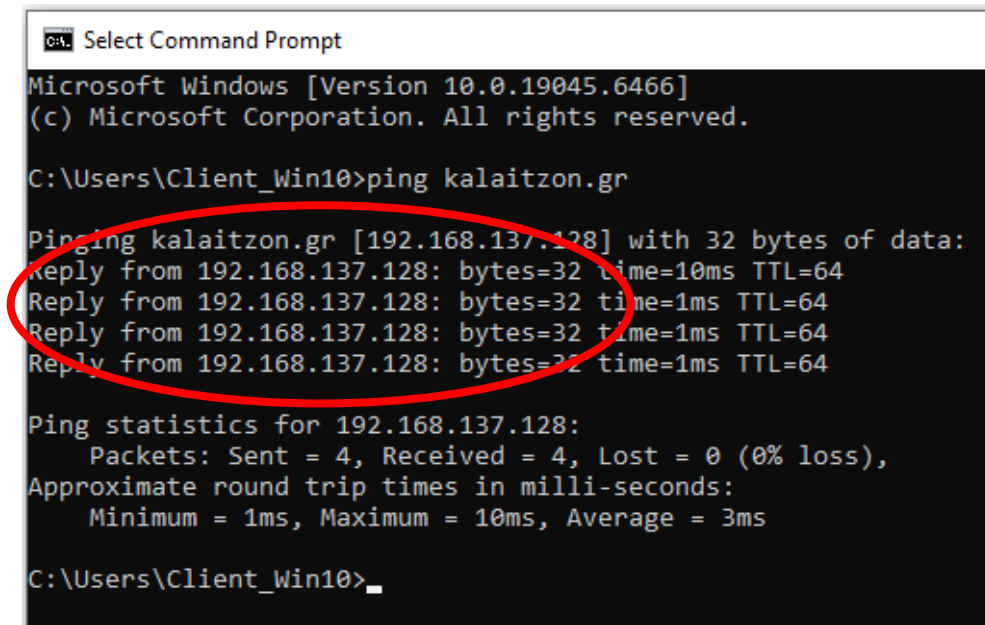
Then, to complete the attack, we activate the plugin `dns_spoof`. Its activation is confirmed by the appearance of the asterisk (\*) to the left of its name. Whereas with ARP Poisoning we were just able to track the movement (Sniffing), now with `dns_spoof` Kali Linux is actively involved in the process. Specifically, the tool monitors the flow of packets and as soon as it detects a DNS request concerning `kalaitzon.gr` domain, it intervenes immediately. Instead of allowing the legitimate server to respond, it sends the victim a spoofed reply with the attacker's IP (.128), thus redirecting the traffic.

Finally, to verify the attack, we go to the victim's computer (Windows 10) and open the browser. Typing in `www.kalaitzon.gr` address, we notice that the page appears with the message "YOU HAVE BEEN HACKED!". This fact confirms that the DNS Spoofing attack worked. Notably, when the victim searched for the domain, the attacker (Kali Linux) intervened in the process. Through Ettercap, it sent a spoofed response before the legitimate DNS server could respond. As a result, the victim's browser was incorrectly routed to the attacker's IP (.128), loading the malicious webpage hosted on Kali's Apache Web Server, instead of the actual content.



In addition, to confirm the attack at the network level, the command was executed `ping kalaitzon.gr` at the victim's command line (CMD). As seen in the image, the system resolves the domain name to the IP address `192.168.137.128`, which corresponds to the attacker's machine (Kali Linux). If the attack had failed, the command would return the IP `192.168.137.129` of the legitimate DNS Server. This result proves that the DNS Spoofing process was successful, as the victim is incorrectly

routed to the malicious node.



```

C:\> Select Command Prompt

Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Client_Win10>ping kalaitzon.gr

Pinging kalaitzon.gr [192.168.137.128] with 32 bytes of data:
Reply from 192.168.137.128: bytes=32 time=10ms TTL=64
Reply from 192.168.137.128: bytes=32 time=1ms TTL=64
Reply from 192.168.137.128: bytes=32 time=1ms TTL=64
Reply from 192.168.137.128: bytes=32 time=1ms TTL=64

Ping statistics for 192.168.137.128:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 10ms, Average = 3ms

C:\Users\Client_Win10>_

```

## Bibliography

Udacity. (2015, February 23). *Apache Web Server* [Video file]. Retrieved from <https://www.youtube.com/watch?v=2lCR0FGLn2c>

Flavor Of The Month. (2020, November 2). *Install & set up Apache Web Server on Windows 10 - quickly!* [Video file]. Retrieved from <https://www.youtube.com/watch?v=oJnCEqeAsUk>

Loi Liang Yang. (2019, December 14). *Get Usernames and Passwords with Ettercap, ARP Poisoning (Cybersecurity)* [Video file]. Retrieved from <https://www.youtube.com/watch?v=CW0Mf9qGBOc>

NetworkChuck. (2020, November 16). *how Hackers SNIFF (capture) network traffic // MiTM attack* [Video file]. Retrieved from <https://www.youtube.com/watch?v=-rSqbgI7oZM>

Chris Greer. (2021, December 9). *How ARP poisoning works // Man-in-the-Middle* [Video file]. Retrieved from <https://www.youtube.com/watch?v=cVTUeEoJgEg>

HackerSploit. (2018, February 15). *MITM With Ettercap - ARP Poisoning* [Video file]. Retrieved from <https://www.youtube.com/watch?v=3UD738uE7Tg>

The Linux Point. (2021, April 16). *Kali Linux - Man in the Middle Attack (MITM) tutorial using Ettercap* [Video file]. Retrieved from <https://www.youtube.com/watch?v=LEPEk5pFffw>

Google Gemini. (n.d.). Retrieved from <https://gemini.google.com/app>